

OPERATING SYSTEMS (UE24CS242B)

Unit 2 – IPC, Threads and Concurrency

Lecture 4 - Threads

Overview and Motivation

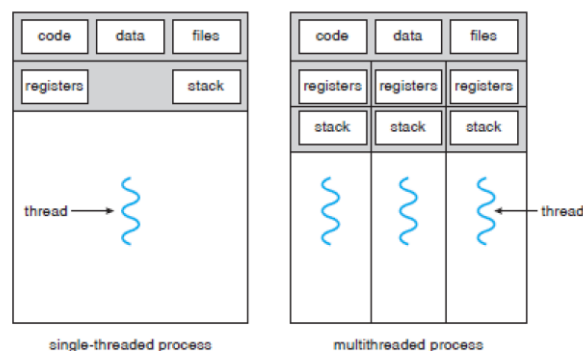
A thread is the fundamental unit of CPU utilization in a modern operating system. It represents a single sequence of execution within a process. Each thread consists of a thread ID, a program counter, a register set, and a stack. While threads within the same process have their own execution context, they share the process's code section, data section, open files, and signals. Because of this shared environment, threads are lighter and more efficient than processes.

Most modern applications are multithreaded. Instead of performing all tasks sequentially, applications divide work among multiple threads. This improves responsiveness, performance, and resource utilization.

Single-Threaded and Multithreaded Processes

In a single-threaded process, only one thread of control exists. If that thread blocks due to an I/O operation or long computation, the entire process stops progressing.

In contrast, a multithreaded process contains multiple threads executing concurrently within the same process. If one thread is blocked, other threads can continue executing. This design significantly improves responsiveness and efficiency.



Single-threaded vs multithreaded process (one thread vs multiple threads sharing memory)

Examples of Multithreaded Applications

A common example is a web browser. Each browser tab may be managed by separate threads. Some threads handle user input, others load web content, fetch data from the network, play videos, or display animations. Even if one thread is waiting for data from the internet, the browser remains responsive to the user.

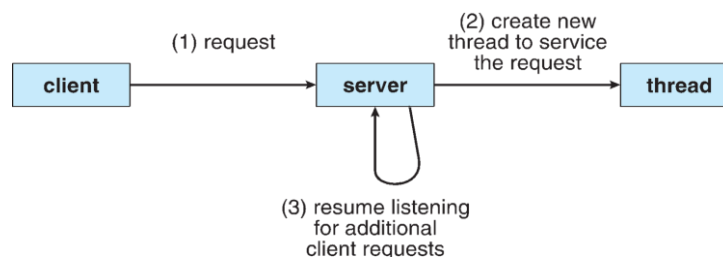
Another example is a word processor, which may use separate threads to:

- Accept user input
- Perform spell checking
- Perform grammar checking

This allows background tasks to run without interrupting the user's typing.

Multithreaded Server Architecture

In a multithreaded server, the server process creates a new thread for each client request. This allows the server to handle multiple clients concurrently instead of serving them one at a time. Threads simplify server design, reduce response time, and improve throughput.



Multithreaded server with multiple client requests handled by separate threads

Process Creation vs Thread Creation

Process creation is considered heavy-weight because it involves:

- Creating a new address space
- Duplicating resources
- High overhead

Thread creation, on the other hand, is light-weight. Threads share the process address space and most resources, so creating and switching between threads is much faster than processes.

Threads are also extensively used in Remote Procedure Call (RPC) systems and operating system kernels. Most modern kernels themselves are multithreaded.

Benefits of Multithreading

Multithreading provides several important benefits:

Responsiveness

Multithreading allows an application to remain responsive even if part of it is blocked or performing a lengthy operation. For example, a GUI application can continue responding to user input while another thread performs background computation or I/O.

Resource Sharing

Threads share the memory and resources of their process. This makes communication between threads easier than communication between processes, which requires IPC mechanisms like shared memory or message passing.

Economy

Threads are cheaper to create and manage than processes. Thread switching requires less overhead than process context switching. For example, in Solaris, creating a process is significantly slower than creating a thread.

Scalability

Multithreading allows applications to take advantage of multiprocessor and multicore architectures. Different threads can run simultaneously on different cores, improving parallel performance.

Processes vs Threads

A process does not share memory by default and has its own file descriptors, filesystem context, and signal handling.

A thread, by default, shares memory, open file descriptors, filesystem context, and signal handling with other threads of the same process.

This sharing is what makes threads efficient, but it also introduces synchronization challenges.

Attributes Shared by Threads

Threads within the same process share many attributes, including:

- Process ID and parent process ID
- Process group ID and session ID
- Controlling terminal
- User and group credentials
- Open file descriptors
- Signal dispositions
- File system information (current working directory, root directory, umask)
- Resource limits
- CPU time consumed
- Nice value

Attributes Specific to Threads

Each thread maintains its own:

- Thread ID
- Signal mask
- Thread-specific (private) data
- errno variable
- Floating-point environment
- Stack (local variables and function call information)
- Register context saved during function calls

These attributes ensure that each thread can execute independently while still sharing the process environment.

Thread Concepts

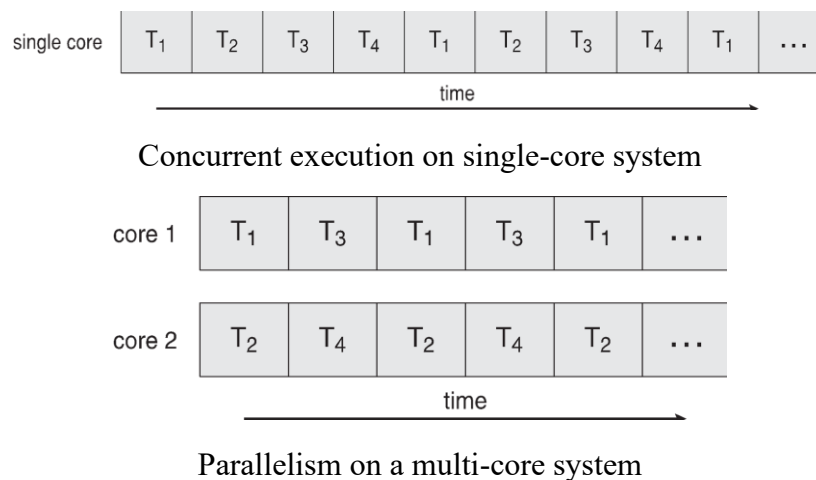
A thread represents an execution context within a process. It includes all information required for execution, such as its ID, register values, stack, scheduling priority and policy, signal mask, and thread-specific data. Thread-specific data allows each thread to maintain private information without synchronizing access with other threads.

When a thread is created, there is **no guarantee which thread will run first**—the newly created thread or the calling thread. The new thread inherits the process address space, floating-point environment, and signal mask, but any pending signals are cleared.

Concurrency vs Parallelism

Concurrency refers to multiple tasks making progress over time. On a single-core system, concurrency is achieved by rapidly switching between tasks using the scheduler.

Parallelism refers to multiple tasks executing **at the same time**, which is possible only on multicore or multiprocessor systems.



Multicore Programming Challenges

Modern multicore systems introduce challenges for programmers, including:

- Dividing activities into parallel tasks
- Balancing workload across cores
- Splitting data efficiently
- Managing data dependencies
- Testing and debugging concurrent programs

Concurrency allows progress, while parallelism provides actual speedup.

Types of Parallelism

Data Parallelism

Data parallelism divides the same data set into subsets and distributes them across multiple cores, performing the same operation on each subset.

Task Parallelism

Task parallelism distributes different threads across cores, with each thread performing a different operation.

As the number of threads increases, CPU architectures provide more support in the form of multiple cores and hardware threads.

Amdahl's Law

Amdahl's Law describes the theoretical speedup of an application when additional cores are added. If a portion of the program must be executed serially, it limits the overall performance improvement.

If **S** is the serial portion of the program and **N** is the number of cores, the speedup is bounded by $1/S$. This means that even with infinite cores, performance improvement is limited by the serial part of the application.

This highlights the importance of minimizing serial sections in parallel programs.

User Threads and Kernel Threads

User threads are managed by user-level thread libraries without kernel involvement. Common thread libraries include:

- POSIX Pthreads
- Windows threads
- Java threads

Kernel threads are supported directly by the operating system kernel. Almost all modern operating systems, including Linux, Windows, Solaris, and macOS, support kernel threads.